چشمک‌زن کردن LED با تایمر و وقفه

آموزش قدم‌به‌قدم و خط‌به‌خط کد تصویر، به زبان خیلی ساده

این برنامه برای میکروکنترلر LPC17xx است. یک LED به پایه‌ی P1.29 وصل شده و برنامه بدون استفاده از تأخیر نرم‌افزاری، آن را روشن و خاموش می‌کند.

شمارش تایمر	رسیدن به عدد مقصد	اجرای تابع وقفه	تغییر LED

چهار بخش اصلی را بشناسیم

نام در کد	معنی ساده	کار آن در این برنامه
LPC_SC	کنترل سیستم	فعال کردن بخش تایمر و ورودی/خروجی و انتخاب کلاک تایمر
LPC_TIM0	تایمر شماره صفر	شمردن پالس‌ها و درخواست وقفه در زمان مشخص
NVIC	مدیریت وقفه پردازنده	اجازه دادن به پردازنده برای پاسخ به درخواست تایمر
LPC_GPIO1	پورت ورودی/خروجی شماره یک	کنترل پایه P1.29 که LED به آن وصل است

مسیر این راهنما

۱. معنی رجیستر، بیت و علامت‌های برنامه را یاد می‌گیریم.

۲. تمام دستورهای تابع اصلی را به ترتیب تصویر بررسی می‌کنیم.

۳. می‌بینیم در زمان وقفه دقیقا چه اتفاقی می‌افتد.

۴. سرعت چشمک را حساب می‌کنیم و کد یک‌جا مرور می‌شود.

> **قرار ما در این آموزش**
>
> هر دستور را اول می‌بینیم، بعد معنی ساده‌اش را می‌خوانیم. شماره صفحات داخل توضیحات تصویر، فقط ارجاع به کتابچه راهنمای همان درس هستند و اثری بر اجرای برنامه ندارند.

قبل از کد: چند مفهوم خیلی ساده

این چند مفهوم را که بدانیم، خواندن بقیه برنامه آسان می‌شود.

رجیستر یک خانه کنترلی داخل میکرو است. هر بیت آن مثل یک کلید کوچک است: صفر یعنی خاموش و یک یعنی روشن. مثلا یک بیت تعیین می‌کند تایمر روشن باشد و بیت دیگری تعیین می‌کند پایه خروجی باشد.

شماره بیت‌ها از صفر شروع می‌شود؛ بنابراین بیت ۲۹، سی‌امین بیت است. علامت دسترسی به عضو، یعنی پیکان، مشخص می‌کند کدام رجیستر کدام بخش را می‌خواهیم.

```
LPC_TIM0->MR0
// Timer 0  ->  Match Register 0
```

یعنی: رجیستر عدد مقصد شماره صفر، در بخش تایمر صفر.

عبارت	کار آن به زبان ساده
`1 << n`	یک را به خانه بیت n ببر؛ فقط همان بیت یک می‌شود.
`x = value`	مقدار کل رجیستر را با مقدار جدید جایگزین کن.
`x |= mask`	بیت‌های مشخص‌شده را یک کن؛ بقیه بیت‌ها دست‌نخورده بمانند.
`x &= ~mask`	بیت‌های مشخص‌شده را صفر کن؛ بقیه بیت‌ها دست‌نخورده بمانند.
`x ^= mask`	بیت‌های مشخص‌شده را برعکس کن: صفر به یک، یک به صفر.
`x & mask`	فقط بیت‌های موردنظر را جدا کن تا بتوانی آن‌ها را بررسی کنی.
`==`	دو مقدار را مقایسه کن؛ خود رجیستر را تغییر نده.

```
1 << 0  = 1          = 0b00000001
1 << 1  = 2          = 0b00000010
1 << 2  = 4          = 0b00000100
1 << 23 = 8,388,608  // only bit 23 is 1
```

خطوطی که با دو اسلش شروع می‌شوند توضیح برای انسان هستند و اجرا نمی‌شوند. نقطه‌ویرگول پایان دستور را مشخص می‌کند و آکولادهای باز و بسته، بدنه تابع، حلقه یا شرط را گروه‌بندی می‌کنند.

> **تفاوت انتساب مستقیم و تغییر فقط یک بیت**
>
> برای عدد مقصد از انتساب مستقیم استفاده شده تا کل مقدار عوض شود. برای کلیدهای کنترلی معمولا از ترکیب با ماسک استفاده می‌شود تا فقط بیت موردنظر تغییر کند و تنظیمات دیگر از بین نرود.

شروع برنامه و تنظیم کلاک

در این صفحه، برنامه معرفی می‌شود و خود تایمر هنوز شروع به شمارش نکرده است.

شروع | فایل معرفی و تابع اصلی

```
#include "LPC17xx.h"

int main (void)
{
```

فایل سرآیند، نام رجیسترها و ثابت‌های این میکرو را در اختیار کامپایلر می‌گذارد؛ به‌خودی‌خود سخت‌افزار را روشن نمی‌کند. تابع اصلی نقطه شروع برنامه است، ورودی ندارد و آکولاد باز، بدنه آن را آغاز می‌کند.

گام ۱ | بخش تایمر صفر را فعال کن

```
LPC_SC->PCONP |= 1 << 1;
```

بیت ۱ رجیستر کنترل توان را یک می‌کنیم تا بخش تایمر صفر فعال شود. تا وقتی این بخش خاموش باشد، تنظیم و شمارش تایمر نتیجه مطلوبی ندارد.

گام ۲ | کلاک تایمر را انتخاب کن

```
LPC_SC->PCLKSEL0 |= 1 << 2;
```

بیت ۲ را یک می‌کند. برای کلاک تایمر صفر، بیت‌های ۳ و ۲ با هم یک انتخاب دو بیتی با چهار حالت می‌سازند. در این مثال، با فرض صفر بودن بیت ۳، حالت دودویی صفریک انتخاب می‌شود: کلاک تایمر برابر کلاک پردازنده است.

بیت‌های ۳ و ۲	کلاک تایمر صفر
00	یک‌چهارم کلاک پردازنده
01	خود کلاک پردازنده؛ انتخاب این مثال
10	نصف کلاک پردازنده
11	یک‌هشتم کلاک پردازنده

یک شرط کوچک اما مهم

عملگر ترکیبی روشن‌کردن بیت، بیت ۳ را پاک نمی‌کند. این خط وقتی حالت انتخاب‌شده را درست می‌سازد که بیت ۳ از قبل صفر باشد؛ این شرط در مقدار اولیه ریست برقرار است. اگر برنامه قبلا آن را تغییر داده باشد، باید ابتدا هر دو بیت را پاک و سپس حالت دلخواه را تنظیم کرد.

عدد مقصد و رفتار تایمر

حالا مشخص می‌کنیم تایمر تا کجا بشمارد و وقتی رسید چه کاری انجام دهد.

گام ۳ | خط پایان شمارش را تعیین کن

```
LPC_TIM0->MR0 = 1 << 23;
```

رجیستر تطبیق شماره صفر برابر دو به توان بیست‌وسه، یعنی ۸,۳۸۸,۶۸ می‌شود. تایمر از صفر بالا می‌رود؛ وقتی شمارنده با این عدد برابر شد، رویداد تطبیق شماره صفر رخ می‌دهد. عدد بزرگ‌تر یعنی فاصله بیشتر بین تغییرهای وضعیت LED.

گام ۴ | در زمان تطبیق، درخواست وقفه بده

```
LPC_TIM0->MCR |= 1 << 0;
```

بیت صفر رجیستر کنترل تطبیق را یک می‌کند. معنی ساده‌اش این است: «اگر شمارنده به عدد مقصد شماره صفر رسید، پردازنده را خبر کن.» خود پردازنده هنوز در گام ۸ اجازه پذیرش این خبر را می‌گیرد.

گام ۵ | در زمان تطبیق، شمارنده را صفر کن

```
LPC_TIM0->MCR |= 1 << 1;
```

بیت ۱ همان رجیستر را یک می‌کند. با رسیدن به عدد مقصد، شمارنده به صفر برمی‌گردد و مسیر را دوباره شروع می‌کند. بدون این تنظیم، تایمر لزوما در هر تطبیق از صفر آغاز نمی‌کرد.

بیت در رجیستر کنترل تطبیق	اثر تنظیم آن برای تطبیق صفر
0	ایجاد درخواست وقفه هنگام تطبیق
1	صفر کردن شمارنده هنگام تطبیق
2	توقف تایمر هنگام تطبیق؛ در این برنامه فعال نشده

دو کلید مستقل داریم

در این کد، هم «خبر دادن» و هم «صفر شدن» فعال‌اند؛ اما «توقف» فعال نیست. بنابراین تایمر پس از هر تطبیق به شمارش ادامه می‌دهد و چشمک زدن تکرار می‌شود.

ریست تایمر و فعال‌سازی وقفه

اول شمارنده را آماده می‌کنیم، بعد مسیر رسیدگی پردازنده به وقفه را باز می‌کنیم.

گام ۶ | شمارنده را در حالت ریست نگه دار

```
LPC_TIM0->TCR |= 1 << 1;
```

بیت ۱ رجیستر کنترل تایمر را یک می‌کند. تا وقتی این بیت یک باشد، شمارنده در حالت ریست نگه داشته می‌شود و مقدارش صفر است. این کار یک شروع تمیز و قابل پیش‌بینی می‌سازد.

گام ۷ | حالت ریست را رها کن

```
LPC_TIM0->TCR &= ~(1 << 1);
```

ماسک بیت ۱ را مشخص می‌کند؛ نقیض آن، همه بیت‌ها به‌جز بیت ۱ را حفظ می‌کند. نتیجه این است که فقط بیت ریست صفر می‌شود. شمارنده آماده حرکت است، ولی هنوز کلید شروع را نزده‌ایم.

بیت در رجیستر کنترل تایمر	وظیفه
0	کلید فعال‌سازی شمارش؛ یک یعنی تایمر فعال است
1	کلید ریست؛ تا وقتی یک باشد شمارنده در حالت ریست می‌ماند

گام ۸ | پذیرش وقفه تایمر را در پردازنده فعال کن

```
NVIC_EnableIRQ(TIMER0_IRQn);
```

این تابع، خط وقفه تایمر صفر را در کنترل‌کننده وقفه پردازنده فعال می‌کند. نام ثابت شماره وقفه در فایل سرآیند تعریف شده و برای تایمر صفر برابر یک است. شماره وقفه با شماره بیت‌های رجیسترهای تایمر یکی نیست.

> **مثال زنگ در**
>
> تنظیم وقفه در تایمر مثل اجازه دادن به زنگ در است که صدا بدهد. فعال‌سازی در پردازنده مثل این است که ما هم آماده شنیدن زنگ باشیم. هر دو لازم‌اند. در این نقطه هنوز خود تایمر شروع به شمارش نکرده است.

استارت تایمر و آماده‌سازی پایه LED

تا اینجا تنظیمات انجام شده‌اند؛ اکنون شمارش و آماده‌سازی خروجی را انجام می‌دهیم.

گام ۹ | شمارش تایمر را آغاز کن

```
LPC_TIM0->TCR |= 1 << 0;
```

بیت صفر رجیستر کنترل تایمر یک می‌شود. چون بیت ریست را قبلا صفر کرده‌ایم، شمارنده می‌تواند با کلاک انتخاب‌شده بالا برود. وقتی به عدد مقصد برسد، تنظیمات تطبیق اثر خود را می‌گذارند.

گام ۱۰ | بخش ورودی و خروجی را فعال نگه دار

```
LPC_SC->PCONP |= (1 << 15);
```

طبق توضیح این کد، بیت ۱۵ مربوط به کنترل توان بخش ورودی/خروجی عمومی است و برنامه آن را فعال می‌کند. این دستور با گام ۱ فرق دارد: آنجا بیت ۱ برای تایمر بود، اینجا بیت ۱۵ برای بخش پایه‌هاست.

گام ۱۱ | پایه متصل به LED را خروجی کن

```
LPC_GPIO1->FIODIR |= 1 << 29;
```

بیت ۲۹ رجیستر جهت پورت یک را یک می‌کند. در نتیجه پایه P1.29 خروجی می‌شود و می‌توانیم مقدار منطقی آن را تغییر دهیم. منظور، پایه شماره ۲۹ از پورت یک است، نه پایه فیزیکی شماره ۲۹ روی بدنه تراشه.

مقدار بیت جهت پایه	معنی
0	ورودی؛ برای خواندن وضعیت پایه
1	خروجی؛ برای فرمان دادن به پایه

> **خروجی کردن با روشن کردن فرق دارد**
>
> این دستور فقط جهت پایه را مشخص می‌کند و به‌تنهایی فرمان روشن شدن LED نیست. روشن یا خاموش شدن واقعی به مقدار پایه و نحوه سیم‌کشی LED بستگی دارد؛ تغییر وضعیت پایه در تابع وقفه انجام می‌شود.

حلقه بی‌نهایت؛ برنامه کجا می‌ماند؟

تابع اصلی تنظیمات را تمام می‌کند، اما نباید برنامه را رها کند و تمام شود.

گام ۱۲ | برنامه را در حالت اجرای همیشگی نگه دار

```
while (1)
{
    // do nothing
}
```

شرط عدد یک همیشه درست است؛ پس حلقه پایان ندارد. داخل آن دستور اجرایی نداریم و فقط یک توضیح نوشته شده است. بااین‌حال وقفه‌ها همچنان کار می‌کنند: تایمر مستقل می‌شمارد و پردازنده هنگام درخواست وقفه وارد تابع مربوط می‌شود.

پایان صوری تابع اصلی

```
    return 0;
}
```

این خط از نظر ساختار زبان، پایان موفق تابع اصلی را نشان می‌دهد؛ اما چون قبل از آن حلقه بی‌نهایت داریم، در اجرای عادی هیچ‌وقت به آن نمی‌رسیم. آکولاد پایانی هم بدنه تابع اصلی را می‌بندد.

بخش	مسئولیت
تابع اصلی	تنظیم سخت‌افزار و زنده نگه داشتن برنامه
تایمر	شمردن مستقل پالس‌ها و رسیدن به عدد مقصد
مدیریت وقفه	بردن پردازنده به تابع رسیدگی در زمان درخواست
تابع وقفه	پاک کردن پرچم و برعکس کردن وضعیت پایه LED

چرا حلقه خالی کافی است؟

لازم نیست پردازنده در هر دور حلقه مقدار تایمر را بخواند. سخت‌افزار تایمر خودش می‌شمارد و خبر می‌دهد. این حلقه تأخیر زمانی ایجاد نمی‌کند و وقفه‌ها را غیرفعال نمی‌کند؛ فقط نمی‌گذارد تابع اصلی برگردد.

پس در بیشتر زمان، برنامه داخل همین حلقه است؛ هر بار که تطبیق رخ می‌دهد، برای مدت کوتاهی تابع وقفه اجرا می‌شود و سپس اجرای برنامه به همین حلقه برمی‌گردد.

تابع وقفه و بررسی علت آن

این بخش بیرون از تابع اصلی نوشته می‌شود و هنگام درخواست وقفه اجرا می‌شود.

گام ۱۳ | تابع مخصوص وقفه تایمر صفر را تعریف کن

```
void TIMER0_IRQHandler(void)
{
```

این تابع ورودی ندارد و مقدار بازگشتی هم ندارد. ما آن را در تابع اصلی صدا نمی‌زنیم؛ هنگام وقفه، پردازنده خودش وارد آن می‌شود. نام تابع باید با نماد جدول بردار وقفه در فایل راه‌اندازی هماهنگ باشد تا آدرس درست به پردازنده داده شود.

در میکرو، جدول بردار وقفه فهرست آدرس تابع‌های رسیدگی است. وقتی وقفه رخ می‌دهد، پردازنده از آن فهرست می‌فهمد باید به کدام تابع برود. آکولاد باز در این خط، بدنه تابع وقفه را شروع می‌کند.

گام ۱۴ | آیا واقعا تطبیق شماره صفر رخ داده است؟

```
if ( (LPC_TIM0->IR & 0x01) == 0x01 )
{
```

رجیستر پرچم‌های وقفه را می‌خوانیم و با ماسک یک، فقط بیت صفر را جدا می‌کنیم. سپس نتیجه را با یک مقایسه می‌کنیم. اگر برابر باشد، یعنی پرچم تطبیق شماره صفر روشن است و باید به آن رسیدگی کنیم.

```
0x01 = 1 = 0b00000001

IR bit 0 = 1  ->  MR0 match interrupt is flagged
IR bit 0 = 0  ->  this particular flag is not set
```

چرا علت وقفه را بررسی می‌کنیم؟

یک تایمر می‌تواند چند علت وقفه داشته باشد؛ مانند تطبیق‌های دیگر یا رویداد گرفتن مقدار شمارنده. پرچم‌ها مشخص می‌کنند کدام رویداد رخ داده است. در این برنامه فقط تطبیق صفر را فعال کرده‌ایم، ولی بررسی پرچم، رسیدگی را دقیق و روشن می‌کند.

پاک کردن پرچم و تغییر وضعیت LED

این دو خط، کار اصلی هر چشمک را انجام می‌دهند.

گام ۱۵ | درخواست رسیدگی‌شده را پاک کن

```
LPC_TIM0->IR |= 1 << 0;
```

هدف این خط پاک کردن پرچم تطبیق صفر است. در این رجیستر، نوشتن یک روی بیت پرچم، آن را پاک می‌کند؛ نوشتن صفر اثری بر آن ندارد. یعنی به سخت‌افزار می‌گوییم: «این درخواست را دیدم؛ آن را تسویه کن.»

روی بیت پرچم چه می‌نویسیم؟	نتیجه در این رجیستر
0	اثری ندارد؛ پرچم باقی می‌ماند
1	پرچم پاک می‌شود

گام ۱۶ | فقط پایه LED را برعکس کن

```
LPC_GPIO1->FIOPIN ^= 1 << 29;
```

عملگر برعکس‌کننده، بیت ۲۹ را هدف می‌گیرد: اگر یک باشد صفر می‌شود و اگر صفر باشد یک می‌شود. ماسک باعث می‌شود سایر بیت‌های پورت هدف تغییر قرار نگیرند. به این کار «تغییر وضعیت» می‌گوییم.

مقدار فعلی بیت پایه	مقدار پس از تغییر وضعیت
0	1
1	0

اگر مدار با یک منطقی LED را روشن کند، یک یعنی روشن و صفر یعنی خاموش. اگر مدار معکوس بسته شده باشد، برعکس است. در هر دو حالت، برنامه وضعیت را عوض می‌کند و LED چشمک می‌زند.

```
    }
}
```

آکولاد اول بدنه شرط و آکولاد دوم بدنه تابع وقفه را می‌بندد. پس از پایان تابع، پردازنده به ادامه اجرای برنامه، یعنی همان حلقه بی‌نهایت، برمی‌گردد.

یک دور کامل و محاسبه زمان چشمک

حالا همه قطعه‌ها را کنار هم می‌بینیم و می‌فهمیم عدد مقصد چه زمانی می‌سازد.

ترتیب	اتفاق
۱	تایمر فعال می‌شود و شمارنده از صفر بالا می‌رود.
۲	شمارنده با عدد مقصد برابر می‌شود؛ رویداد تطبیق رخ می‌دهد.
۳	پرچم وقفه روشن می‌شود و شمارنده برای دور بعد صفر می‌شود.
۴	پردازنده وارد تابع وقفه می‌شود و پرچم تطبیق را بررسی می‌کند.
۵	پرچم را پاک و وضعیت پایه LED را برعکس می‌کند.
۶	به حلقه اصلی برمی‌گردد؛ تایمر همچنان به شمارش ادامه می‌دهد.

محاسبه با فرض صفر بودن پیش‌مقیاس تایمر

```
MR0 = 2^23 = 8,388,608

f_count  = f_timer / (PR + 1)
t_toggle = (MR0 + 1) / f_count
T_blink  = 2 * t_toggle
```

اگر پیش‌مقیاس صفر باشد، سرعت شمارش با کلاک تایمر برابر است. از شروع شمارش در صفر تا آماده شدن برای دور بعد، به اندازه عدد مقصد به‌اضافه یک پالس طول می‌کشد؛ برای همین در رابطه دقیق یک واحد اضافه شده است. در توضیح ساده می‌توان گفت «حدود ۸٫۴ میلیون پالس».

کلاک تایمر فرضی	فاصله هر تغییر وضعیت	یک روشن و خاموش شدن کامل
12 MHz	حدود ۰٫۶۹۹ ثانیه	حدود ۱٫۳۹ ثانیه
100 MHz	حدود ۰٫۰۸۳۹ ثانیه	حدود ۰٫۱۶۷۸ ثانیه

فرکانس واقعی پردازنده در این تصویر معلوم نیست

این کد نسبت کلاک تایمر به کلاک پردازنده را تنظیم می‌کند، اما خود فرکانس پردازنده را تعیین نمی‌کند. برای زمان دقیق باید تنظیمات راه‌اندازی و کلاک برد را بدانیم. همچنین یک چشمک کامل شامل دو تغییر وضعیت است، نه یکی.

پیوست یک: کد تابع اصلی

همان دستورهای تصویر، با فاصله‌گذاری خواناتر؛ توضیح هر خط در صفحه‌های قبل آمده است.

```
#include "LPC17xx.h"

int main(void)
{
    // (1) Timer 0 configuration
    LPC_SC->PCONP |= 1 << 1;
    LPC_SC->PCLKSEL0 |= 1 << 2;

    LPC_TIM0->MR0 = 1 << 23;

    LPC_TIM0->MCR |= 1 << 0;
    LPC_TIM0->MCR |= 1 << 1;

    LPC_TIM0->TCR |= 1 << 1;
    LPC_TIM0->TCR &= ~(1 << 1);

    // (2) Enable timer interrupt
    NVIC_EnableIRQ(TIMER0_IRQn);

    LPC_TIM0->TCR |= 1 << 0;
    LPC_SC->PCONP |= (1 << 15);
    LPC_GPIO1->FIODIR |= 1 << 29;

    while (1)
    {
        // do nothing
    }

    return 0;
}
```

روش مرور سریع

از بالا به پایین بخوانید: فعال کردن تایمر، انتخاب کلاک، تعیین عدد مقصد، تنظیم وقفه و صفر شدن، ریست اولیه، اجازه پذیرش وقفه، شروع شمارش و در پایان آماده کردن پایه LED. تابع وقفه در صفحه بعد آمده است.

پیوست دو: تابع وقفه و جمع‌بندی

این تابع باید بیرون از تابع اصلی قرار بگیرد؛ نام آن برای فایل راه‌اندازی اهمیت دارد.

```
// This name must match the Timer 0 interrupt vector.
void TIMER0_IRQHandler(void)
{
    if ((LPC_TIM0->IR & 0x01) == 0x01)
    {
        LPC_TIM0->IR |= 1 << 0;
        LPC_GPIO1->FIOPIN ^= 1 << 29;
    }
}
```

سه نکته ظریف برای خواندن دقیق کد

۱. انتخاب کلاک: روشن کردن بیت ۲ به‌تنهایی بیت ۳ را صفر نمی‌کند. انتخاب این مثال به مقدار اولیه مناسب آن دو بیت متکی است.

۲. پاک کردن پرچم: رجیستر پرچم با نوشتن یک پاک می‌شود، نه با نوشتن صفر. عبارت اصلی، یک عملیات خواندن، تغییر و نوشتن است؛ اگر پرچم دیگری هم یک باشد، ممکن است آن هم با نوشته شدن یک پاک شود. در این مثال فقط پرچم تطبیق صفر را فعال کرده‌ایم.

```
// Clear only the MR0 flag, without reading other flags:
LPC_TIM0->IR = 1 << 0;
```

۳. زمان چشمک: مقدار عدد مقصد به‌تنهایی زمان را مشخص نمی‌کند؛ کلاک تایمر و پیش‌مقیاس هم لازم‌اند. مثال‌های زمانی این راهنما با پیش‌مقیاس صفر حساب شده‌اند.

> **جمع‌بندی در یک جمله**
>
> تایمر می‌شمارد؛ در عدد مقصد، وقفه می‌سازد و صفر می‌شود؛ پردازنده پرچم را پاک و LED را برعکس می‌کند؛ سپس این چرخه تکرار می‌شود.

مرجع کد: تصویر ارسالی شما. این راهنما برای توضیح آموزشی همان کد تهیه شده است. قلم فارسی: وزیرمتن.